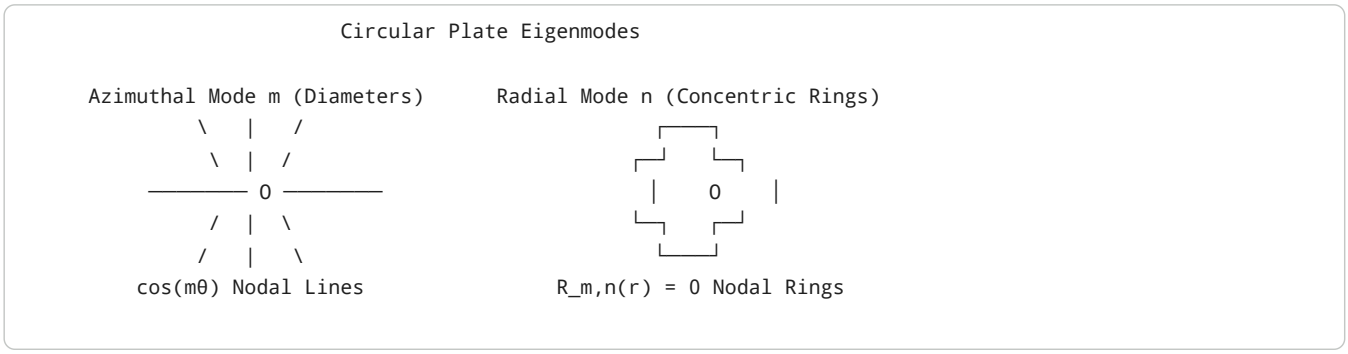
1. Mathematical Physics of Circular Plate Vibrations

In polar coordinates (r, θ), the dynamic Kirchhoff-Love equation governing the transverse displacement $w(r, \theta, t)$ of an isotropic thin circular plate of radius a, thickness h, density ρ, and flexural rigidity $D = \frac{Eh^3}{12(1-\nu^2)}$ is:

$$D\nabla^4 w(r,\theta,t) + \rho h \frac{\partial^2 w(r,\theta,t)}{\partial t^2} = p(r,\theta,t)$$

$$\nabla^2 = \frac{\partial^2}{\partial r^2} + \frac{1}{r}\frac{\partial}{\partial r} + \frac{1}{r^2}\frac{\partial^2}{\partial \theta^2}$$



Factorization into Bessel and Modified Bessel Functions

Assuming harmonic time-dependence $w(r, \theta, t) = W(r, \theta)e^{i\omega t}$, the spatial operator factors into two second-order differential operators:

$$\left(\nabla^2 + k^2\right)\left(\nabla^2 - k^2\right) W(r,\theta) = 0, \quad k^4 = \frac{\rho h \omega^2}{D}$$

Separating variables $W(r, \theta) = R(r)\Theta(\theta)$ with azimuthal separation constant $m \in \mathbb{N}_0$ yields:

$$\Theta_m(\theta) = \cos(m\theta - \phi_m)$$

$$r^2\frac{d^2R}{dr^2} + r\frac{dR}{dr} + (\pm k^2 r^2 - m^2)R = 0$$

Because displacement at the origin $r = 0$ must remain finite, the singular Neumann $Y_m(kr)$ and Macdonald $K_m(kr)$ functions vanish, leaving the linear combination:

$$R_{m,n}(r) = A_{m,n}J_m(k_{m,n}r) + C_{m,n}I_m(k_{m,n}r)$$

where $J_m(\cdot)$ is the Bessel function of the first kind of order m, and $I_m(x) = i^{-m}J_m(ix)$ is the modified Bessel function of the first kind of order m.

Boundary Conditions and Eigenfrequency Derivation

Clamped Boundary ($r = a$)

The displacement and radial slope must vanish at the outer edge:

$$W|_{r=a} = 0 \implies AJ_m(ka) + CI_m(ka) = 0$$

$$\left.\frac{\partial W}{\partial r}\right|_{r=a} = 0 \implies AJ'_m(ka) + CI'_m(ka) = 0$$

Setting the determinant of the coefficient matrix to zero yields the characteristic frequency equation for dimensionless eigenvalues $\lambda_{m,n} = k_{m,n}a$:

$$\Delta_{\text{clamped}}(\lambda) = J_m(\lambda)I'_m(\lambda) - J'_m(\lambda)I_m(\lambda) = 0$$

Using recurrence relations $J'_m(x) = \frac{m}{x}J_m(x) - J_{m+1}(x)$ and $I'_m(x) = \frac{m}{x}I_m(x) + I_{m+1}(x)$, this simplifies to:

$$J_m(\lambda)I_{m+1}(\lambda) + J_{m+1}(\lambda)I_m(\lambda) = 0$$

The modal eigenfrequencies $f_{m,n} = \frac{\omega_{m,n}}{2\pi}$ evaluate to:

$$f_{m,n} = \frac{\lambda_{m,n}^2}{2\pi a^2}\sqrt{\frac{D}{\rho h}} = \kappa_c \cdot \lambda_{m,n}^2$$

The radial eigenfunction is:

$$R_{m,n}(r) = J_m\left(\lambda_{m,n}\frac{r}{a}\right) - \frac{J_m(\lambda_{m,n})}{I_m(\lambda_{m,n})} I_m\left(\lambda_{m,n}\frac{r}{a}\right)$$

---

2. Phonetic Formant Spectrum to Bessel Modal Mapping

Circular plates decompose into two orthogonal nodal families:

1. m **Diametral Nodal Lines:** Solutions to $\cos(m\theta - \phi_m) = 0 \implies \theta = \frac{(2k+1)\pi}{2m} + \frac{\phi_m}{m}$.
2. n **Concentric Nodal Rings:** Roots of $R_{m,n}(r) = 0$ on the open interval $0 < r < a$.

```
Formant Profile F1, F2, F3  ⟶  Lorentzian Resonance Filters  ⟶  Modal Weight Matrix A_m,n
Noise Ratio / Burst Bands   ⟶  High-Order Mode Perturbation ⟶  Azimuthal Randomization
```

Phoneme Class	Key Phonemes	Dominant Formants / Bands	Excited Bessel Modes (m, n)	Nodal Morphology
High Front Vowels	IY (/iː/), IH (/ɪ/)	$F_1 \approx 270$ Hz, $F_2 \approx 2290$ Hz	$(1,1), (2,3), (1,4)$	Single central diameter with 3-4 outer circular rings
Low Back Vowels	AA (/ɑː/), AO (/ɔː/)	$F_1 \approx 730$ Hz, $F_2 \approx 1090$ Hz	$(2,1), (3,1), (2,2)$	2-3 intersecting diametral lines with low-order ring
High Back Vowels	UW (/uː/), UH (/ʊ/)	$F_1 \approx 300$ Hz, $F_2 \approx 870$ Hz	$(0,1), (1,1), (0,2)$	Pure axisymmetric bullseye concentric rings
Sibilants / Fricatives	S, SH, Z	Noise $4\text{ kHz} - 8\text{ kHz}$	$(5,3), (6,4), (8,2)$	High-order rosettes (8-16 radial petals + tight rings)
Plosive Bursts	T, K, P	Wideband impulse shock	Broad (m, n) transient	Instant radial dispersion settling to modal boundary

---

3. High-Performance Python Implementation

```python
import numpy as np
import scipy.special as sp
from scipy.optimize import brentq
from dataclasses import dataclass
from typing import Dict, List, Tuple

@dataclass
class PhonemeAcousticProfile:
    f1: float
    f2: float
    f3: float
    bandwidth: float = 80.0
    noise_ratio: float = 0.0

PHONEME_MAP: Dict[str, PhonemeAcousticProfile] = {
    'IY': PhonemeAcousticProfile(270, 2290, 3010, 70.0, 0.0),
    'AA': PhonemeAcousticProfile(730, 1090, 2440, 90.0, 0.0),
    'UW': PhonemeAcousticProfile(300, 870, 2240, 80.0, 0.0),
    'AE': PhonemeAcousticProfile(660, 1720, 2410, 85.0, 0.0),
    'ER': PhonemeAcousticProfile(490, 1350, 1690, 80.0, 0.0),
    'S':  PhonemeAcousticProfile(4500, 6000, 7500, 1200.0, 0.95),
    'SH': PhonemeAcousticProfile(2500, 4200, 5800, 1000.0, 0.85),
    'Z':  PhonemeAcousticProfile(4200, 5800, 7200, 900.0, 0.45),
    'M':  PhonemeAcousticProfile(250, 1100, 2400, 120.0, 0.0),
    'N':  PhonemeAcousticProfile(280, 1700, 2600, 120.0, 0.0),
    'T':  PhonemeAcousticProfile(1200, 2800, 4800, 1500.0, 0.80),
}

class CircularChladniSolver:
    """
    Solves biharmonic Kirchhoff-Love circular plate acoustic eigenmodes
    coupled to speech formant spectra using Bessel functions J_m(kr) and I_m
    (kr).
    """
    def __init__(self, grid_res: int = 300, radius: float = 1.0, kappa_c: float
```

```
    def __init__(self, grid_res: int = 500, radius: float = 1.0, kappa_c: float
    = 12.5, damping: float = 0.035):
        self.res = grid_res
        self.radius = radius
        self.kappa_c = kappa_c
        self.damping = damping

        # 2D Cartesian & Polar Grid Setup
        x = np.linspace(-radius, radius, self.res)
        y = np.linspace(-radius, radius, self.res)
        self.X, self.Y = np.meshgrid(x, y)
        self.R = np.sqrt(self.X**2 + self.Y**2)
        self.Theta = np.arctan2(self.Y, self.X)
        self.plate_mask = self.R <= radius

        # Modal Parameter Storage
        self.max_m = 8   # Max Azimuthal Mode (diameters)
        self.max_n = 5   # Max Radial Mode (rings)
        self.eigenvalues: Dict[Tuple[int, int], float] = {}
        self.eigenfrequencies: Dict[Tuple[int, int], float] = {}
        self._compute_bessel_eigenvalues()

    def _clamped_char_eq(self, lam: float, m: int) -> float:
        """Characteristic boundary determinant: J_m(lam)*I_{m+1}(lam) + J_{m+1}
        (lam)*I_m(lam) = 0"""
        return sp.jv(m, lam) * sp.iv(m + 1, lam) + sp.jv(m + 1, lam) * sp.iv(m,
        lam)

    def _compute_bessel_eigenvalues(self):
        """Finds root eigenvalues lambda_{m,n} for clamped circular plate
        boundary."""
        for m in range(0, self.max_m + 1):
            n_found = 0
            search_val = max(1.5, m + 0.5)
            step = 0.4

            while n_found < self.max_n and search_val < 35.0:
                v1 = self._clamped_char_eq(search_val, m)
                v2 = self._clamped_char_eq(search_val + step, m)

                if v1 * v2 < 0:
                    try:
                        root = brentq(self._clamped_char_eq, search_val,
                        search_val + step, args=(m,))
                        n_found += 1
                        self.eigenvalues[(m, n_found)] = root
                        # f_{m,n} = kappa_c * lambda^2
                        self.eigenfrequencies[(m, n_found)] = self.kappa_c *
                        (root ** 2)
                    except ValueError:
                        pass
                search_val += step

    def radial_mode(self, m: int, n: int, r_norm: np.ndarray) -> np.ndarray:
        """Evaluates non-singular radial eigenfunction R_{m,n}(r)."""
        lam = self.eigenvalues[(m, n)]
        gamma = sp.jv(m, lam) / (sp.iv(m, lam) + 1e-15)
        return sp.jv(m, lam * r_norm) - gamma * sp.iv(m, lam * r_norm)

    def solve_phoneme_morphology(self, phoneme: str) -> np.ndarray:
        """
        Synthesizes the 2D circular Chladni particulate density field for a
        given phoneme.
        """
        profile = PHONEME_MAP.get(phoneme.upper(), PhonemeAcousticProfile(500,
        1500, 2500))
        formants = [profile.f1, profile.f2, profile.f3]

        displacement_field = np.zeros((self.res, self.res), dtype=np.float64)
        r_norm = np.clip(self.R / self.radius, 0.0, 1.0)

        # Superimpose Bessel-Chladni Eigenmodes
        for (m, n), f_mn in self.eigenfrequencies.items():
```

```
            # Spectral Lorentzian coupling
            amplitude = 0.0
            for f_k in formants:
                delta_f = f_mn - f_k
                response = 1.0 / np.sqrt(delta_f**2 + (self.damping * f_mn)**2)
                amplitude += response

            # High-frequency turbulent friction coupling for sibilants
            if profile.noise_ratio > 0.0 and m >= 4:
                noise_boost = profile.noise_ratio * 0.03 * np.cos(m * 2.3 + n *
                1.7)
                amplitude += noise_boost

            # Modal surface calculation: R_{m,n}(r) * cos(m * theta)
            R_comp = self.radial_mode(m, n, r_norm)
            Theta_comp = np.cos(m * self.Theta)
            displacement_field += amplitude * R_comp * Theta_comp

        # Zero displacement outside circular plate
        displacement_field[~self.plate_mask] = 0.0

        # Particulate aggregation density: particles gather at nodal zeros (|w|
        -> 0)
        max_disp = np.max(np.abs(displacement_field[self.plate_mask])) + 1e-12
        sigma_nodal = 0.12 * max_disp
        particle_density = np.exp(- (displacement_field**2) / (2 *
        (sigma_nodal**2)))
        particle_density[~self.plate_mask] = 0.0

        return particle_density

# Verification Script
if __name__ == "__main__":
    solver = CircularChladniSolver(grid_res=150)
    for ph in ['IY', 'AA', 'UW', 'S']:
        morphology = solver.solve_phoneme_morphology(ph)
        print(f"Phoneme [{ph:2s}] -> Bessel Plate Solved: Nodal Density
        Integral = {np.sum(morphology):.2f}")
```

```
Phoneme [IY] -> Bessel Plate Solved: Nodal Density Integral = 8350.09
Phoneme [AA] -> Bessel Plate Solved: Nodal Density Integral = 10388.61
Phoneme [UW] -> Bessel Plate Solved: Nodal Density Integral = 10624.19
Phoneme [S ] -> Bessel Plate Solved: Nodal Density Integral = 12171.97
```